LAYOUT-PRESERVING OCR EDITOR

Research & Development Project — Final 2 Months of Internship

Project idea in one line: Take a physically available book, handwritten/printed notes, scanned page, or document image/PDF and convert it into an editable digital document while keeping the original structure and layout as close as possible.

Item	Details
Duration	8 weeks / approximately 2 months
Domain	OCR + Computer Vision + Document Understanding + Web Application
Repository	Private GitHub repository
Deployment	Working public demo using a free/low-cost hosting option where practical
Main output	Editable document that visually follows the source page, not just plain extracted text

This is deliberately an ambitious project. Similar problems are being worked on in the industry and research community, and there is no expectation that the first version will solve every document perfectly. The purpose is to see how far you can take the problem with the time, tools and knowledge available to you.

The most important part is not simply using an OCR library. The interesting engineering problem is what happens after text has been detected: understanding where the text belongs on the page, reconstructing the layout, making it editable and keeping the editable version visually close to the source.

1. What We Want to Build

Build a web application that allows a user to upload a scanned book page, notes, printed document, image or PDF and then work with the converted version in an editor.

A typical screen should look conceptually like this:

LEFT SIDE	RIGHT SIDE
Original Uploaded image/PDF page. It should remain visible so the user can compare the source with the converted document.	Editable version A document editor that contains the extracted content in editable form while trying to preserve the original page structure, positions, formatting and elements.

The user should be able to look at the original and the reconstructed document side by side and make corrections where OCR or layout detection is not perfect.

JETHAT CYBER SECURITY

Example

Suppose a page contains a heading at the top, two columns of text, an image in the middle, a caption below the image and a paragraph at the bottom. A normal OCR application may return one long block of text. That is not the main goal here.

The target is to identify those elements and create an editable representation where the heading remains a heading, the columns remain columns, the image stays approximately where it belongs, the caption stays with the image, and the text remains editable.

2. The Real Problem to Solve

OCR itself is only one part of the problem. The project should explore the complete process of turning pixels into a structured, editable document.

1. Read the input image/PDF and separate pages if necessary.
2. Detect useful page regions such as text, headings, images, tables, captions, lists and other elements.
3. Recognize the text inside those regions.
4. Determine reading order — especially for multi-column pages.
5. Estimate layout information such as bounding boxes, alignment, spacing and relative positions.

6. Reconstruct the page in an editable format.
7. Render the editable result in the browser so that the user can modify it.
8. Allow the user to compare the reconstructed page with the original.
9. Provide a practical way to correct OCR/layout mistakes.
10. Export the corrected result into a useful editable format where feasible.

3. What Makes This Different From a Basic OCR Project?

- Plain OCR output is not enough. The project must preserve as much page structure as reasonably possible.
- The intern needs to investigate layout detection and reading order, not only text recognition.
- The editable result should be usable by a person rather than being a raw OCR JSON response.
- The application should support visual comparison between source and reconstructed content.
- The intern should measure how close the reconstructed output is to the source and identify where it fails.
- The project should be designed so that additional document types and better models can be added later.

Important: There are existing OCR/document-AI products and open-source projects that address parts of this problem. You are allowed to study them. The goal is not to pretend the problem has never been solved; the goal is to understand the problem, build your own working pipeline and see what level of result you can achieve.

JETHAT CYBER SECURITY
अनुक्षणं रक्षामहे

4. Research Questions

During the project, you should investigate questions such as:

- How accurately can text regions be detected on different types of pages?
- How can reading order be determined for one-column and multi-column documents?
- Which OCR approach works best for the selected document types?
- Can preprocessing improve recognition of poor scans, faded pages or photographs?
- How can font size, alignment, spacing and text blocks be estimated from an image?
- How can tables, images, captions and headings be represented in an editable document?
- Which browser editor model is suitable for representing the reconstructed page?
- How should the system handle OCR mistakes without destroying the original layout?
- What is the best compromise between visual similarity and editability?

> You do not need to answer every question perfectly. Pick a realistic subset and go deeper rather than trying to build everything.

5. Suggested System Flow

Upload → Page detection → Image preprocessing → Layout analysis → OCR → Reading-order analysis → Document reconstruction → Editable editor → Export

The architecture can be different if you have a better idea. The HLD should explain your final approach.

6. Minimum Product Requirements

Module	Minimum expectation
Upload	Image and PDF upload with validation.
Page handling	Process single and multi-page documents, at least for the selected scope.
Original viewer	Original image/PDF remains visible for comparison.
OCR	Extract text from the selected document types.
Layout detection	Detect meaningful regions and preserve their relative arrangement.

Reading order	Handle normal reading order and investigate multi-column layouts.
Editable output	Show reconstructed content in a browser-based editor.
Side-by-side comparison	Original on one side and editable reconstruction on the other.
Correction	Users should be able to correct extracted text/layout where required.
Export	Export to at least one useful editable representation such as HTML, DOCX, JSON or another justified format.
Deployment	Working public demo with frontend/backend connected.

7. Features That Can Be Added If Time Allows

- Table reconstruction.
- Image extraction and placement.
- Automatic heading detection.
- Page numbering and headers/footers.
- Multiple language support.
- Handwritten text support as an experimental feature.
- Document history/versioning.
- Zoom and synchronized scrolling between original and editor.
- Click an editable text block and highlight its corresponding region on the original image.
- Confidence-based highlighting of areas that may need manual correction.
- Export to DOCX/PDF while trying to retain the reconstructed layout.

8. HLD Requirements

Before building the complete system, submit an HLD containing:

- Problem statement and scope.
- Architecture diagram.
- Frontend/backend/AI processing components.
- Upload-to-editor data flow.
- OCR and layout processing pipeline.

- Representation of page elements and bounding boxes.
- How the reconstructed document will be represented in the editor.
- API definitions and sample responses.
- Database/storage plan, if required.
- Processing strategy for large or multi-page documents.
- Error handling and retry approach.
- Security considerations for uploaded files.
- Deployment architecture.
- Technology choices and alternatives considered.

One important design question to address

How will you represent a page internally? For example, will you store text blocks with coordinates, paragraphs, columns, images and styles as structured JSON and then render that structure in the editor? Your HLD should explain your approach.

9. Research & Evaluation

Because the final goal is visual and structural reconstruction, do not evaluate the project only by OCR text accuracy.

Evaluation	Possible measurement
OCR quality	CER / WER or another suitable text metric
Layout quality	Region/bounding-box overlap, alignment or another justified measure
Reading order	Correct ordering of blocks/columns on a test set
Visual similarity	Compare source and reconstructed rendering using a justified image/layout metric or human evaluation
Editability	Can a user easily modify text without breaking the page structure?
Processing time	Average time per page/document

Create a small benchmark set containing different page types: clean printed pages, old/scanned pages, multi-column pages, pages with images, tables or unusual formatting. Explain your selection.

10. Technology Choices

You are free to select the stack, but every major technology choice should have a reason. Open-source tools and models are preferred where practical so the project can be developed and demonstrated without depending entirely on paid APIs.

Area	Possible choices
Frontend	React / Next.js / HTML-CSS-JS / similar
Editor	TipTap, ProseMirror, Lexical, Slate, CKEditor or another suitable editor
Backend	FastAPI / Flask / Django / similar
OCR	Tesseract, PaddleOCR, EasyOCR, docTR, transformer-based OCR or another justified approach
Vision/layout	OpenCV and/or suitable layout/document models
Storage	SQLite/PostgreSQL/object storage or another justified choice
Hosting	Vercel/GitHub Pages for frontend where appropriate + free-tier backend option

You may research current open-source OCR and document-understanding projects for ideas. Do not make the project simply a wrapper around an existing service. The work should show your own pipeline, engineering decisions, integration and evaluation.

11. Eight-Week Plan

Week	Focus	Expected outcome
1	Research & scope	Study existing approaches, select document types, prepare sample dataset and define success criteria.
2	HLD & baseline	Complete HLD and build a basic OCR + layout prototype.
3	Layout research	Experiment with blocks, reading order, columns and page coordinates.
4	Reconstruction	Create the first editable reconstruction of a page.
5	Editor + comparison	Build the side-by-side original/editor experience and correction workflow.
6	Multi-page + robustness	Improve handling of different pages and document structures.
7	Evaluation & improvement	Benchmark results, analyse failures and improve the pipeline.

8	Deployment & final demo	Deploy, document, complete research report and present the system.

12. Final Deliverables

Deliverable	What should be submitted
Private GitHub repository	Complete source code, frontend, backend, AI/OCR pipeline, README and meaningful commit history.
Live web application	Public URL where the evaluator can upload sample documents and test the main workflow.
HLD	Final architecture, data flow, APIs, document representation, security and deployment.
Research report	Existing approaches studied, experiments, methodology, benchmark, results and limitations.
Sample dataset/benchmark	Description of documents used for testing and evaluation.
Evaluation report	OCR accuracy, layout/reconstruction observations, visual comparison and processing time.
Setup guide	How another developer can run the project locally.
Final presentation	10–15 minute explanation + live demo.

13. Evaluation

Area	Weight	What we will look at
Research & problem solving	25%	Depth of investigation, experiments, reasoning and learning from failures.
OCR/layout reconstruction	25%	How well the system converts a real page into a structured editable version.
Software engineering	20%	Code quality, architecture, APIs, testing and maintainability.
HLD & technical understanding	15%	Ability to explain the complete design and technical decisions.

UI/deployment	10%	Usability, side-by-side editor experience and working online demo.
Ownership/presentation	5%	Communication, initiative and ability to defend the work.

14. A Note About the Difficulty of This Project

This is intentionally not a small assignment. You may find that some parts work easily while other parts are much harder than expected. That is part of the project.

There are already strong OCR and document-understanding systems available, and some research/open-source projects are working on similar problems. That does not make this project unnecessary. The purpose here is to see whether you can understand a difficult problem, break it into smaller parts, experiment with different solutions and build a working prototype.

> If you cannot achieve perfect layout reconstruction in two months, that is okay. A well-designed prototype that clearly shows what works, what does not work and how you would improve it can be a very strong outcome.

15. Internship Completion & Future Opportunity

The project will be considered as an important part of the final internship evaluation. Subject to satisfactory completion of the internship requirements and company policy, an Internship Certificate and, where appropriate, an LOR may be provided.

We are also using this project to understand who is comfortable working on difficult AI/ML problems. Interns who show strong research ability, technical ownership, problem-solving skills and the ability to build beyond a basic prototype may be considered for future AI projects. If a suitable full-time opportunity becomes available, HR may contact the relevant interns separately. This is not a guarantee of employment.

16. Final Submission Checklist

- Private GitHub repository shared with the evaluator.

- The live demo is accessible.
- Original document/image is visible alongside the editable result.
- Text is actually editable in the browser.
- Layout reconstruction works for the selected document types.
- Research experiments and results are documented.
- Limitations and failed cases are documented honestly.
- HLD matches the final implementation.
- README contains setup instructions.
- No passwords/API keys/secrets are committed.
- Final presentation and live demonstration are ready.

Final thought

Don't try to impress us by adding dozens of features. Try to impress us by solving the core problem properly. If you can take a real physical page, understand it, reconstruct it and give a user an editable version that looks and behaves close to the original, you will have built something genuinely interesting.

Good luck. Take ownership of the problem and see how far you can take it.